Del AVR al ESP32

El ESP32 no tiene DDRB, PORTB ni PINB, pero **la idea no cambia**: un pin se controla escribiendo bits en **registros mapeados en memoria**. Cambian los nombres, el ancho (32 bits) y aparece una «central» que conecta pines con periféricos.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Leer-modificar-escribir: se pierde un cambio

PORTB |= (1 << PB5) son 3 pasos

1. `loop()` **lee (`in`)** — copia el registro — `copia = 0010 0000`
2. **Interrupción** — cambia el registro **entre medio** — `PORTB → 0010 0001`
3. `loop()` **calcula (`eor`)** — con la copia **vieja** — `resultado 0000 0000`
4. `loop()` **escribe (`out`)** — el resultado viejo — `PORTB ← 0000 0000`
5. **El cambio se perdió** — el de la interrupción. Con **dos núcleos** el riesgo es mayor.

W1TS Y W1TC: ESCRITURA ATÓMICA (SIN LEER ANTES)

REGISTRO	BITS EN 1	BITS EN 0
W1TS (*write 1 to set*)	Se **encienden**	No se tocan
W1TC (*write 1 to clear*)	Se **apagan**	No se tocan

Una sola escritura, **sin leer antes**: la operación es **atómica** y nada se pierde.

```
GPIO_OUT antes    0 1 0 1
escribo en W1TS   0 0 1 0
GPIO_OUT después  0 1 1 1
```

BLINK CON REGISTROS DEL ESP32

```
#include "soc/gpio_reg.h"
constexpr uint8_t PIN_LED = 16;
constexpr uint32_t MASK_LED = 1UL << PIN_LED;

void setup() {
  pinMode(PIN_LED, OUTPUT);                 // configura
}
void loop() {
  REG_WRITE(GPIO_OUT_W1TS_REG, MASK_LED);   // como PORTB |=
  delay(500);
  REG_WRITE(GPIO_OUT_W1TC_REG, MASK_LED);   // como PORTB &= ~
  delay(500);
}
```

LED en GPIO16 con 330 Ω a GND · placa «ESP32 Dev Module» · core Espressif 3.x · pinMode() configura; los registros leen y escriben.

2 De digitalWrite al pin: las capas

arriba, más fácil · abajo, más control · la API cuesta tiempo (AP1 D)

1. **Tu código** — `digitalWrite(16, HIGH)`
2. **Core Arduino-ESP32** — llama a ESP-IDF
3. **ESP-IDF (Espressif)** — `gpio_set_level(16, 1)`
4. **Registro** — `GPIO_OUT_W1TS_REG ← 1 << 16`
5. **Hardware** — el pin 16 sube a **3,3 V**

3 Del reset al `loop()`

el programa que nunca termina: no existe un «fin»

1. **Reset** — arranca con la alimentación
2. **Bootloader ROM** — lee los pines de **strapping**
3. **2.º bootloader** — está en la flash
4. `app_main()` — ESP-IDF
5. `loopTask` — FreeRTOS, núcleo 1
6. `setup()` — configura **una vez**
7. `loop()` — repite su ronda **siempre**; solo termina si se corta la energía

`loop() se repite: leer entradas → decidir → escribir salidas`

ANTES DE SETUP()

Las globales inicializadas se copian de la flash a la SRAM. **Todo sketch del ESP32 ya corre sobre FreeRTOS** (Unidad 2).

LO QUE ARDUINO ESCONDE (AVR)

`main()` llama a `init()` (timers, ADC…), `setup()` y un `for (;;) { loop(); }`. En el ESP32, `loopTask` hace lo mismo.

LO QUE CUESTA UN LOOP() VACÍO

- **Activo** a 240 MHz: decenas de mA, aunque `loop()` no haga nada.
- **Deep sleep**: casi todo se apaga y un temporizador lo despierta: del orden de µA.

MAPA DE PINES DEL ESP32 DEVKIT V1 · SEMÁFORO

Seguros usar estos

16 17 18 19 21 22 23 25 26 27 32 33

Para LEDs y pulsadores. En la AP2: LEDs en 16, 17, 18, 19 y pulsador en 21.

Evitar strapping

0 2 5 12 15

El chip los lee al arrancar: **no conectar entradas**. GPIO0 en bajo = modo de carga (botón BOOT) · GPIO2 debe estar bajo o al aire (LED azul de la placa) · **GPIO12 en alto: la placa no arranca**.

Con cuidado solo entrada

34 35 36 VP 39 VN

Solo lectura y **sin pull-up interna**: usar pull-up externa (10 kΩ).

Prohibido no usar

1 TX0 3 RX0 6–11 flash

UART0 carga el programa y el Monitor Serie; 6–11 son la flash y no salen en esta placa.

WOKWI NO REPRODUCE ESTAS FALLAS

Los problemas de strapping solo aparecen con la **placa real**.

LÍMITES A 3,3 V

`R = (3,3 V − Vf) / I` · ≤ **10 mA** por pin · **no toleran 5 V**: un sensor de 5 V directo puede dañar el pin.

DIAGNÓSTICO: CUANDO EL PROGRAMA NO SUBE

SÍNTOMA	CAUSA PROBABLE	SOLUCIÓN
No aparece ningún puerto	Cable solo de carga o falta driver	Otro cable · driver CP210x/CH340
Wrong boot mode / Timed out	No entró en modo de carga	Mantener **BOOT** en «Connecting...»
Access denied	Puerto ocupado por otra ventana	Cerrar el otro Monitor Serie
Caracteres basura	Velocidad distinta	115200 en `Serial.begin` y en el Monitor
Brownout detector	Alimentación débil	Otro puerto USB o cable
No arranca con el circuito	Pin de strapping forzado	Mover a un pin seguro

RESPUESTAS · PREDICCIÓN

01 *Predicción: global, local y `static` se incrementan en cada `loop()`. ¿Qué imprime la 3.ª pasada?* `global=3 local=1 static=3`. La **local** se crea de nuevo en cada vuelta (vale 1 siempre); la **global** y la **static** se crean una sola vez y conservan su valor.